```
    )
    load_row
    @concert.broadcast_schedule_change
  end

  private def load_data
    @user = current_user
    @cart = ShoppingCart.find_or_create_by(user_id: params[:user_id])
    @concert = Concert.find(params[:concert_id])
  end

  private def load_row
    @row = @concert.row_at(
      tickets: @concert.tickets_in_row(params[:row_number].to_i),
      number: params[:row_number].to_i,
      tickets_to_buy_count: params[:tickets_to_buy_count].to_i
    )
  end
end
```

Both of these actions have the same structure: they grab their data and update the shopping cart, which uses previously written code to update Ticket records in the database to their new purchased or unpurchased state.

They make two ActionCable broadcasts: one to any page open to the schedule that the number of tickets available has changed (I've offloaded that one to Concert):

appendix_hotwire/01/app/models/concert.rb

```
def broadcast_schedule_change
  ActionCable.server.broadcast(
    "schedule",
    {concerts: [{concertId: id, ticketsRemaining: tickets.unsold.count}]}
  )
end
```

and one to any other page opened to the same concert that the ticket status has changed. I've offloaded that one to Ticket, using an ActiveRecord callback, so that the change happens any time a ticket is updated:

appendix_hotwire/01/app/models/ticket.rb